

Token-Based Algorithms

In token-based algorithms, a unique token is shared among the sites. A site is allowed to enter its CS if it possesses the token.

Token-based algorithms use sequence numbers instead of timestamps.

- (Used to distinguish between old and current requests.)
- In token-based algorithms, A site is allowed to enter its critical section if it possesses the unique token. Non-token based algorithms uses
- timestamp to order requests for the critical section where as sequence number is used in token based algorithms.

Each requests for critical section contains a sequence number. This sequence number is used to distinguish old and current requests.

Suzuki-Kasami's Broadcast Algorithm

- If a site wants to enter the CS and it does not have the token, it broadcasts a REQUEST message for the token to all other sites.
- A site which possesses the token sends it to the requesting site upon the receipt of its REQUEST message.
- If a site receives a REQUEST message when it is executing the CS, it sends the token only after it has completed the execution of the CS.

Suzuki-Kasami algorithm is a token-based algorithm for achieving mutual exclusion in distributed systems. This is a modification of Ricart-Agrawala algorithm, a permission based (Non-token based) algorithm which uses REQUEST and REPLY messages to ensure mutual exclusion.

Suzuki-Kasami's Broadcast Algorithm

Data structure and Notations:

An array of integers $RN[1...N]$

- A site S_i keeps $RN_i[1...N]$, where $RN_i[j]$ is the largest sequence number received so far through REQUEST message from site S_i .

An array of integer $LN[1...N]$

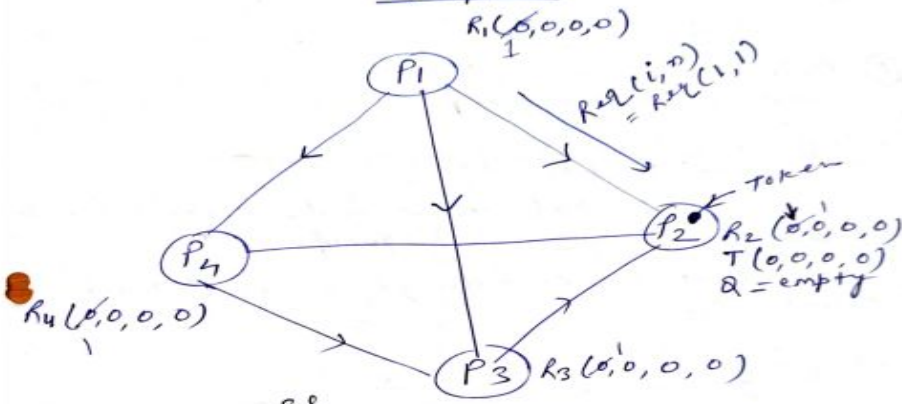
- This array is used by the token. $LN[j]$ is the sequence number of the request that is recently executed by site S_j .

A queue Q

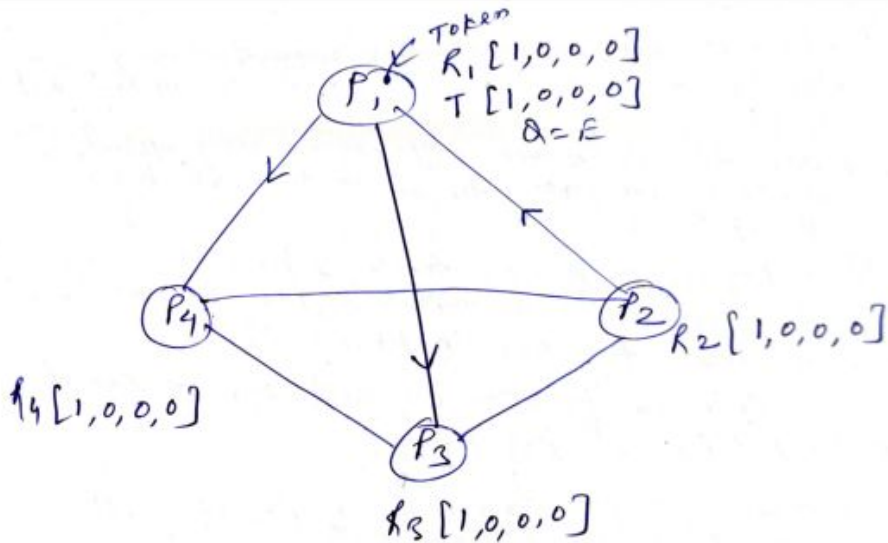
This data structure is used by the token to keep record of ID of sites waiting for the token

Suzuki-Kasami's Broadcast Algorithm

Example



Suzuki-Kasami's Broadcast Algorithm



The Algorithm

Requesting the critical section

- (a) If requesting site S_i does not have the token, then it increments its sequence number, $RN_i[i]$, and sends a REQUEST(i, sn) message to all other sites. (sn' is the updated value of $RN_i[i]$.)
- (b) When a site S_j receives this message, it sets $RN_j[i]$ to $\max(RN_j[i], sn)$. If S_j has the idle token, then it sends the token to S_i if $RN_j[i] = LN[i] + 1$.

Executing the critical section

- (c) Site S_i executes the CS after it has received the token.

The Algorithm

Releasing the critical section Having finished the execution of the CS, site S_i takes the following actions:

- (d) It sets $LN[i]$ element of the token array equal to $RN_i[i]$.
- (e) For every site S_j whose id is not in the token queue, it appends its id to the token queue if $RN_i[j] = LN[j] + 1$.
- (f) If the token queue is nonempty after the above update, S_i deletes the top site id from the token queue and sends the token to the site indicated by the id.

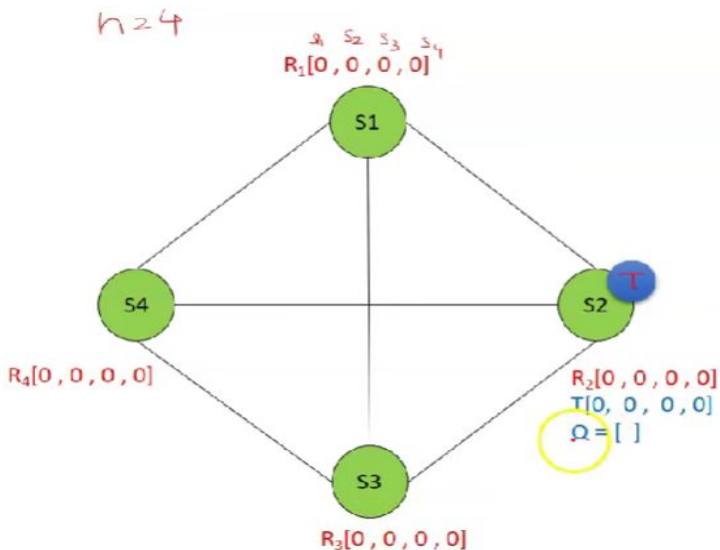
The Algorithm

Suzuki Kasami Algorithm

Requesting Stage:

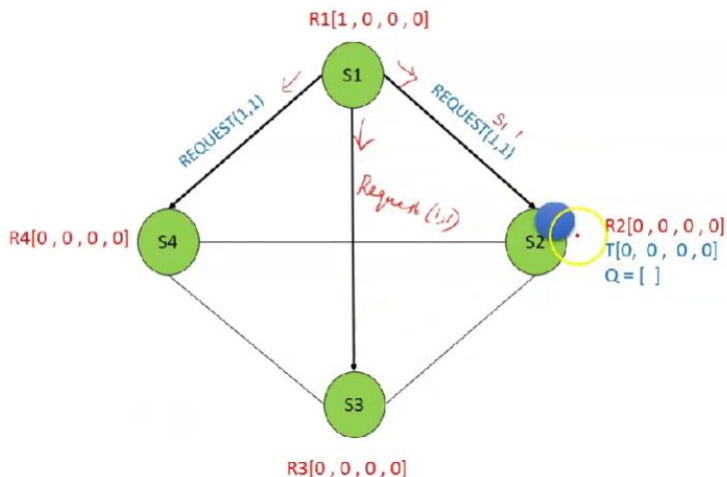
- Site S_i wants to enter into CS:
- Sends a token request message $REQUEST(i,n)$ to all other sites.
- On receiving the Request message from S_i , site S_j update its request R_j with $\max(R_j, n)$
- On receiving S_i 's request, the 2 conditions to be checked:
 - a. **Check for outdated request**
if $R_j[i] < n$, then the request is not an outdated request
 - b. **Determining outstanding request**
if $R_j[i] = T[i] + 1$, then there is no outstanding request

The Algorithm Initial State



The Algorithm

S1 wants to enter CS and broadcast token request **REQUEST(1,1)**

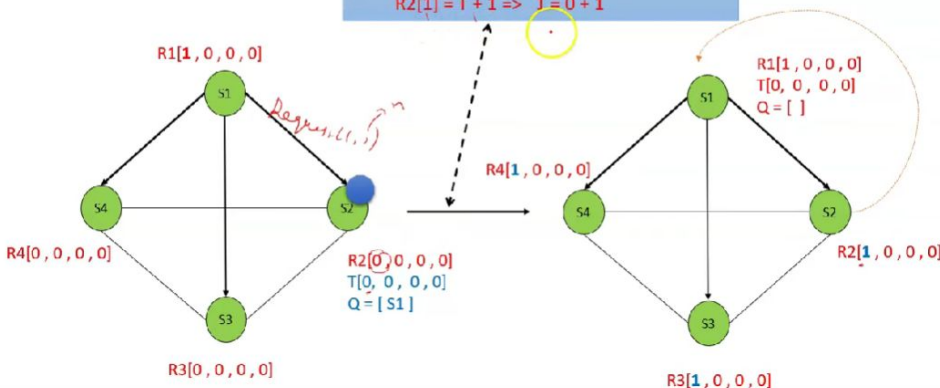


The

Algorithm

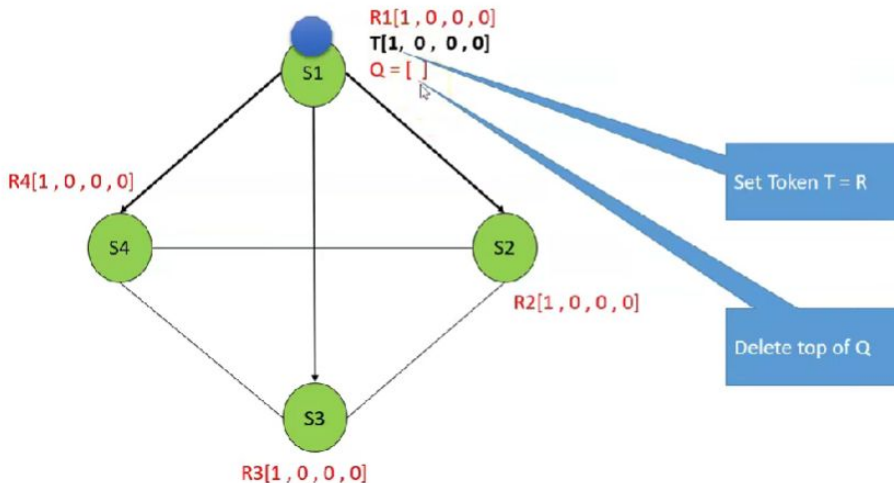
On receiving S1's request

1. Check for outdated request
 $R_2[1] = 0$ and $n=1$
 $n > R_2[1]$, so not an outdated request
2. Determining outstanding request
 $R_2[1] = T + 1 \Rightarrow 1 = 0 + 1$



The

Site S1, on completing CS

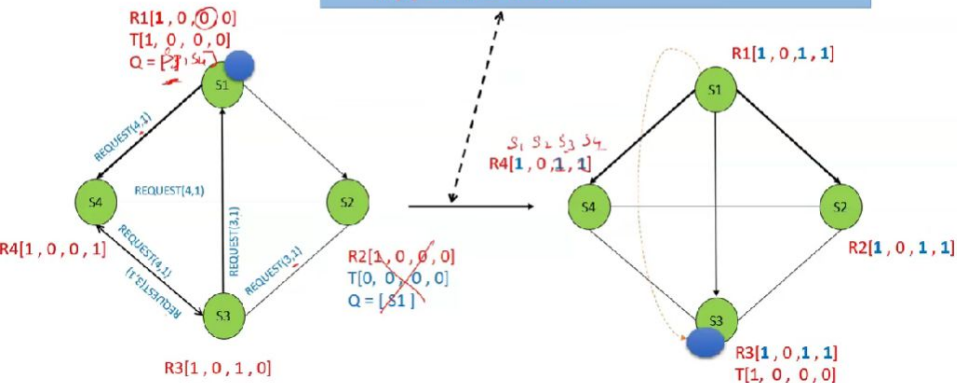


The Algorithm

Next S3 and S4 Simultaneously requesting CS

top of queue is S3, the 2 conditions to be checked:

1. Check for outdated request
 $R1[3] = 0$ and $n=1$, $n > R1[3]$, so not an outdated request
2. Determining outstanding request
 $R1[3] = T + 1 \Rightarrow 1 = 0 + 1$

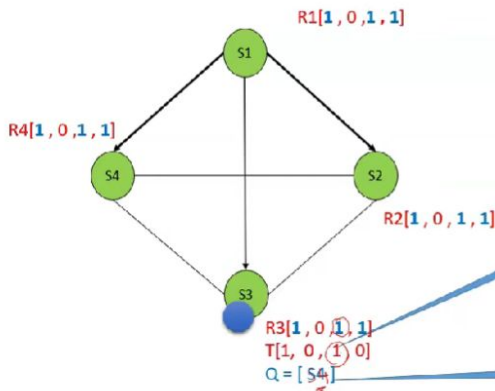


The ...

Site S3, on completing CS

Queue top deleted

On completion of CS by S3, it sets $T3 = R3$



Set Token $T = R$

Delete top of Q

Message Complexity:

The algorithm requires 0 message invocation if the site already holds the idle token at the time of critical section request or maximum of N message per critical section execution. This N messages involves

-
- (N – 1) request messages
1 reply message

Drawbacks of Suzuki-Kasami Algorithm:

Non-symmetric Algorithm: A site retains the token even if it does not have requested for critical section. According to definition of symmetric algorithm "No site possesses the right to access its

- critical section when it has not been requested."



Performance:

- Synchronization delay is 0 and no message is needed if the site holds the idle token at the time of its request.
- In case site does not holds the idle token, the maximum synchronization delay is equal to maximum message transmission time and a maximum of N message is required per critical section invocation.

Raymond's Tree-Based Algorithm

- Token – based
- Sites are logically arranged as a directed tree such that the edges of the tree are having directions toward the site(Tree Root) that has the token.
- Every site has a local variable called holder that points to an immediate neighbour node on a directed path to the root node. Holder of root is self pointed.
- Every site keeps a FIFO queue called request Q that stores the requests sent by neighbouring site to this site.

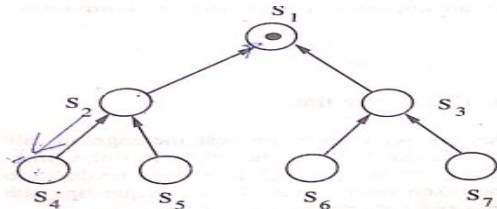
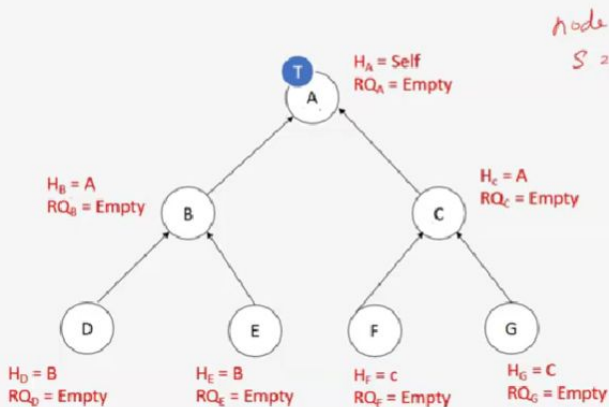


FIGURE 6.12
Sites arranged in a tree configuration.

Raymond's Tree-Based Algorithm

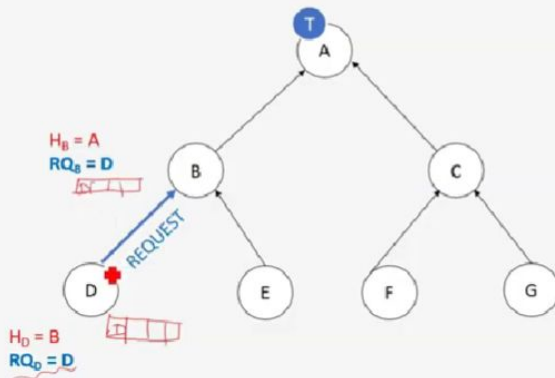
Raymond-tree Example: *Initial state*



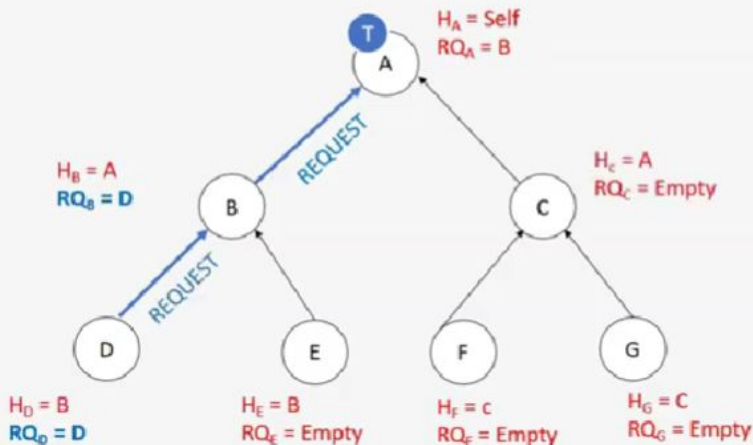
Raymond's Tree-Based Algorithm

Raymond-tree Example: *D send request for token to enter into CS*

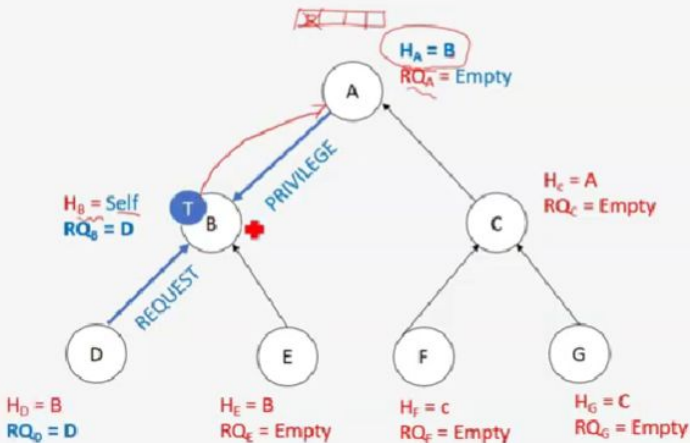
REQUEST PRIVILEGE



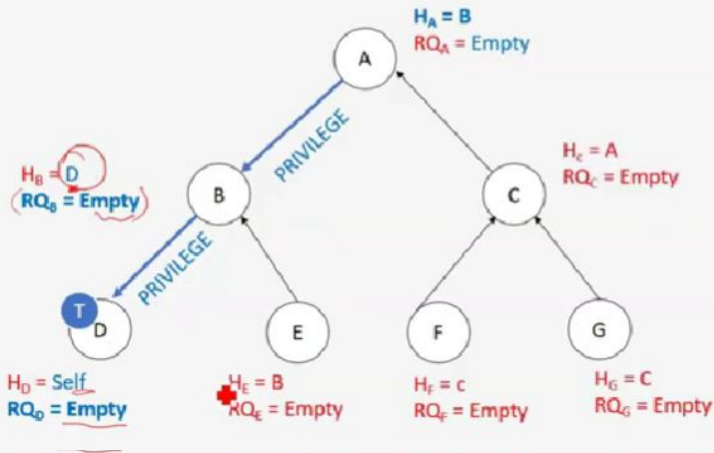
Raymond-tree Example:



Raymond-tree Example:



Raymond-tree Example:



Raymond's Tree-Based

The Algorithm : Requesting the Critical Section

When a site wants to enter CS,

1. it sends REQUEST to its immediate parent, It then update its request Q
2. When the parent receives this REQUEST message, it update its Request Q and forwarded the REQUEST message to its parent.
3. This directed REQUEST forwarding continues till Root reached.
4. On receiving the REQUEST message, the root node sends the token to the site from which it received REQUEST with PRIVILEGE message and sets its holder value to the requesting site. Hence direction changed.
5. When a site receives the token, it deletes the top entry from request Q, sends the token to the indicated site and sets its holder variable to point the target site.

Raymond's Tree-Based Algorithm

Executing the CS

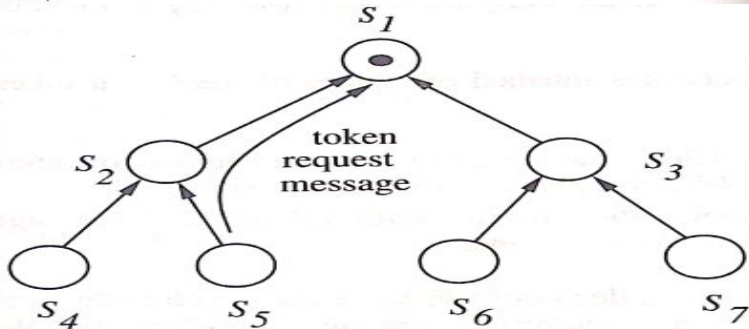
6. A site enters CS when it receives the token and its own entry is at the top of the request Q, then deletes the top entry from its own request Q and enters the CS.

Releasing the CS

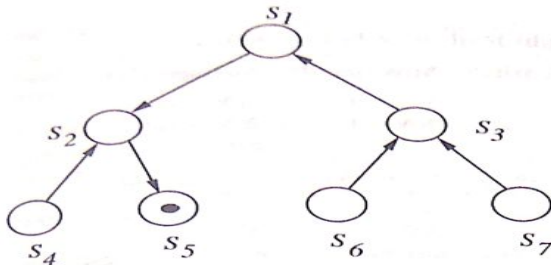
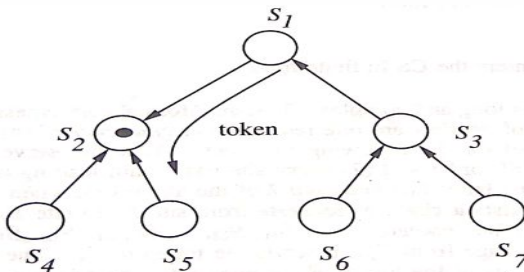
After executing CS,

7. If its request Q is non-empty, then deletes the top entry from queue and sends the token to that site, and sets the holder value to the target node.
8. If the request Q is empty, then the site sends a REQUEST message to the site which is pointed at by the holder variable.

Example 6.4. In Fig. 6.13, site S_5 sends a REQUEST message to S_2 , which propagates it to the root S_1 . Root S_1 sends the token to S_2 which in turn sends the token to S_5 (Fig. 6.14). The token travels along the same path traveled by the REQUEST message (but in the opposite direction) and it also reverses the direction of the edges on the path. Consequently, the site that executes the CS last becomes the new root. (See Fig. 6.15.)



Raymond's Tree-Based Algo



Performance-Raymond's Tree-Based Algorithm

- The average message complexity of Raymond
- algorithm is $O(\log N)$ because the average distance
- between any two nodes in a tree with N nodes is
- $O(\log N)$.

Performance comparison of token and non-token based algorithm

TABLE 6.1

A comparison of performance (*ll* = light load, *hl* = heavy load)

NON-TOKEN	Resp. time (<i>ll</i>)	Sync Delay	Messages (<i>ll</i>)	Messages (<i>hl</i>)
Lamport	$2T+E$	T	$3(N-1)$	$3(N-1)$
Ricart-Agrawala	$2T+E$	T	$2(N-1)$	$2(N-1)$
Maekawa	$2T+E$	$2T$	$3\sqrt{N}$	$5\sqrt{N}$
TOKEN	Resp. time (<i>ll</i>)	Sync Delay	Messages (<i>ll</i>)	Messages (<i>hl</i>)
Suzuki and Kasami	$2T+E$	T	N	N
Singhal's Heuristic	$2T+E$	T	$N/2$	N
Raymond	$T(\log N)+E$	$T \log(N)/2$	$\log(N)$	4

Performance-Raymond's Tree-Based Algorithm

-
- <https://www.youtube.com/watch?v=Uy8hI-Gqjc4>
- https://www.youtube.com/watch?v=aWne_qIR2X
- |
- <https://www.youtube.com/watch?v=1zPFuEvSFKk>